

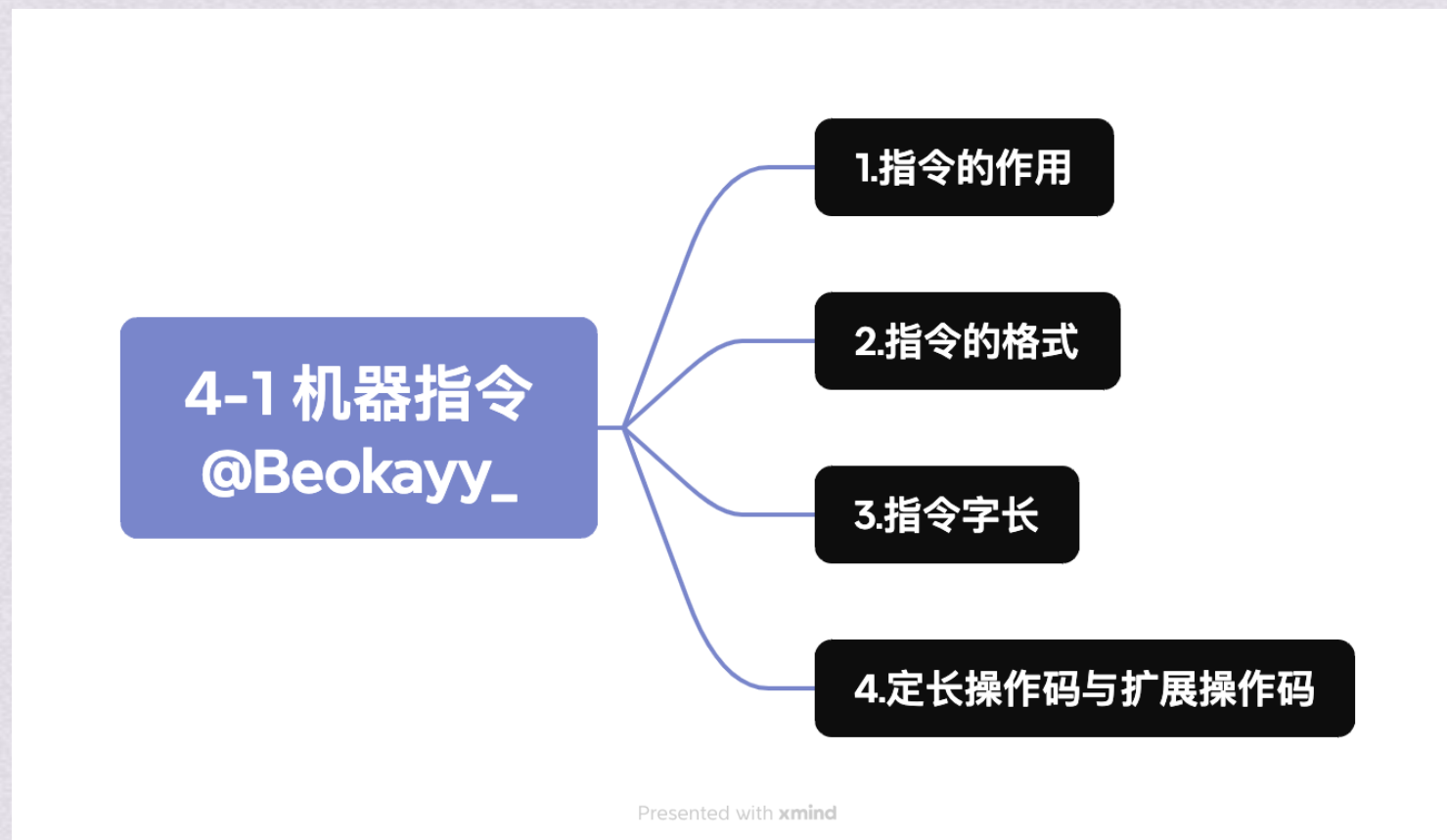
bok25

**基
础**

班、计算机组成原理



机器指令





1.指令的作用



机器指令

1.指令的定义与作用

冯诺依曼机

计算机就是连续执行每一条机器指令而实现全自动工作的

做一道番茄炒鸡蛋

进程

切番茄、打鸡蛋

开火倒油

炒菜

放调料

出锅装盘

指令1

指令2

指令3

指令4

指令5

进程是指令的集合



机器指令

1.指令的定义与作用



计算机活着就是为了执行指令！
执行一条条的机器指令就可以完成想要的操作。
全部机器指令的集合称为机器的指令系统



2.指令的格式

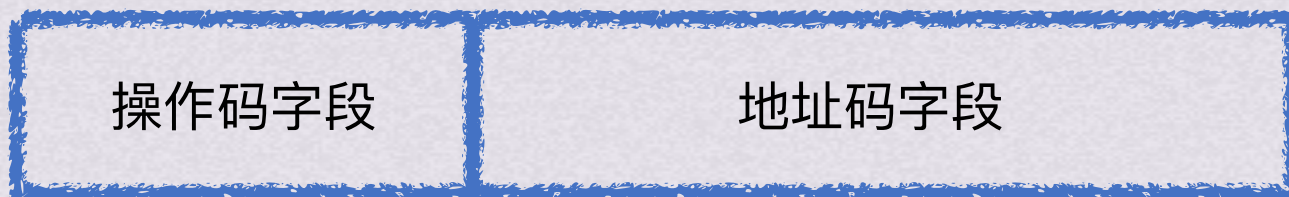


机器指令

2.指令的格式

指令的基本格式

指令是由机器码和地址码两部分组成的





机器指令

2.指令的格式



指令的基本格式

指令是由机器码和地址码两部分组成的

操作码字段

操作码用来指明该指令所要完成的操作

操作码位数反映了机器的操作种类，即机器允许的指令条数

操作码占 n 位，则该机器最多包含 $2^n=128$ 条指令

操作码长度可以固定也可以变化

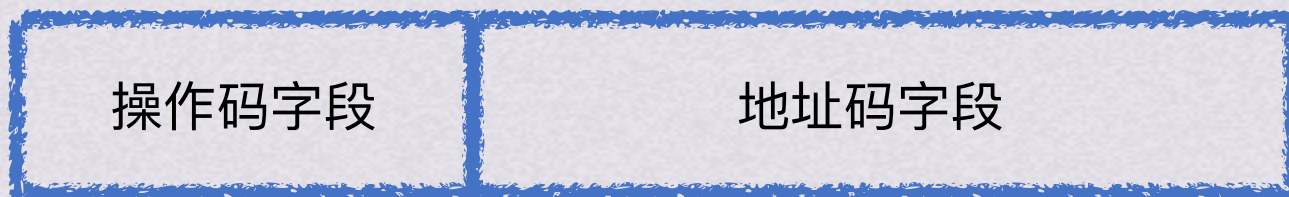


机器指令

2.指令的格式

指令的基本格式

指令是由机器码和地址码两部分组成的





机器指令

2.指令的格式



指令的基本格式

指令是由机器码和地址码两部分组成的

地址码字段

地址码用来指出该指令的源操作数的地址、结果的地址以及下一条指令的地址
这个地址可以是主存地址，也可以是寄存器地址

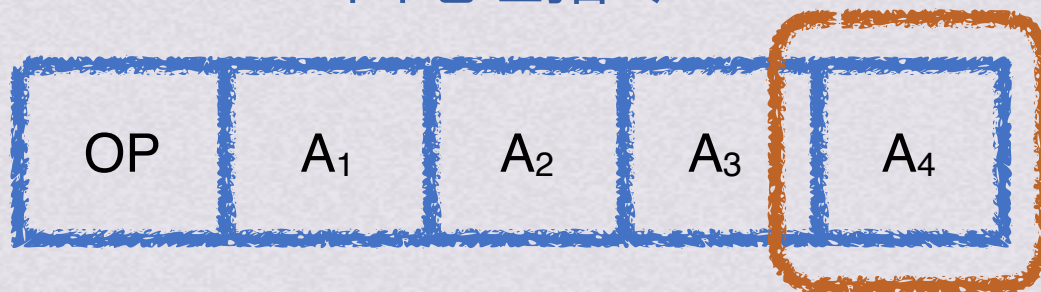


机器指令

2.指令的格式

格式分类

四地址指令



程序中指令一般按顺序执行
PC可以通过自增自动形成
下一条指令的地址
因此A4可省去

$(A_1)OP(A_2) \rightarrow A_3$

A₄指示下一条指令的地址

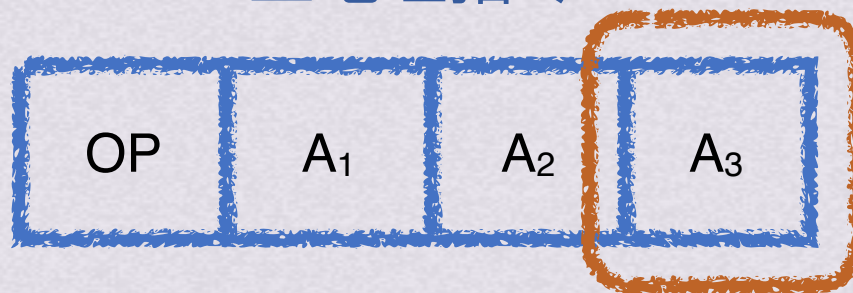


机器指令

2.指令的格式

格式分类

三地址指令



如果指令运算的结果是中间结果
没必要每次都放入主存
可以暂存在CPU的寄存器中

$$(A_1)OP(A_2) \rightarrow A_3$$

后续指令的地址由PC自增得出

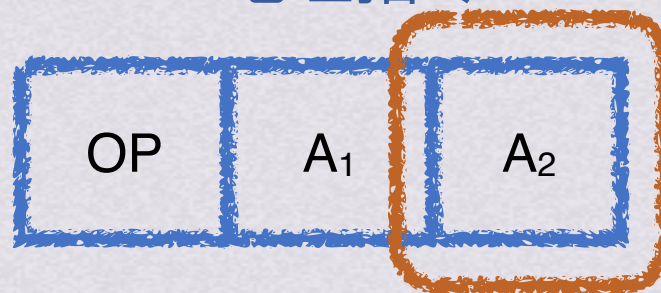


机器指令

2.指令的格式

格式分类

二地址指令



可以将一个源操作数隐含在运算器的ACC中

$(A_1)OP(A_2) \rightarrow A_1$

$(A_1)OP(A_2) \rightarrow A_2$

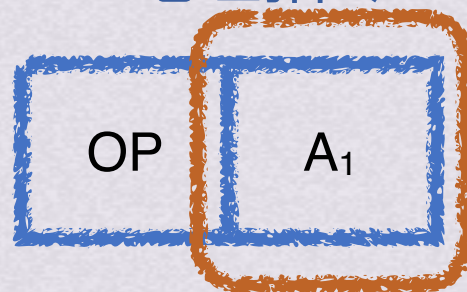


机器指令

2. 指令的格式

格式分类

一地址指令



还可以直接去掉地址字段
形成零地址指令

$(ACC)OP(A_1) \rightarrow ACC$

$(ACC)OP(A_1) \rightarrow A_1$



机器指令

2.指令的格式

格式分类

零地址指令



空操作NOP、停机HLT这类指令只有操作码
子程序返回RET、中断返回IRET这类指令没有地址码，其操作数的地址隐含在堆栈指针SP中



3.指令字长



机器指令

3.指令字长

指令字长



指令的长度即为指令字长

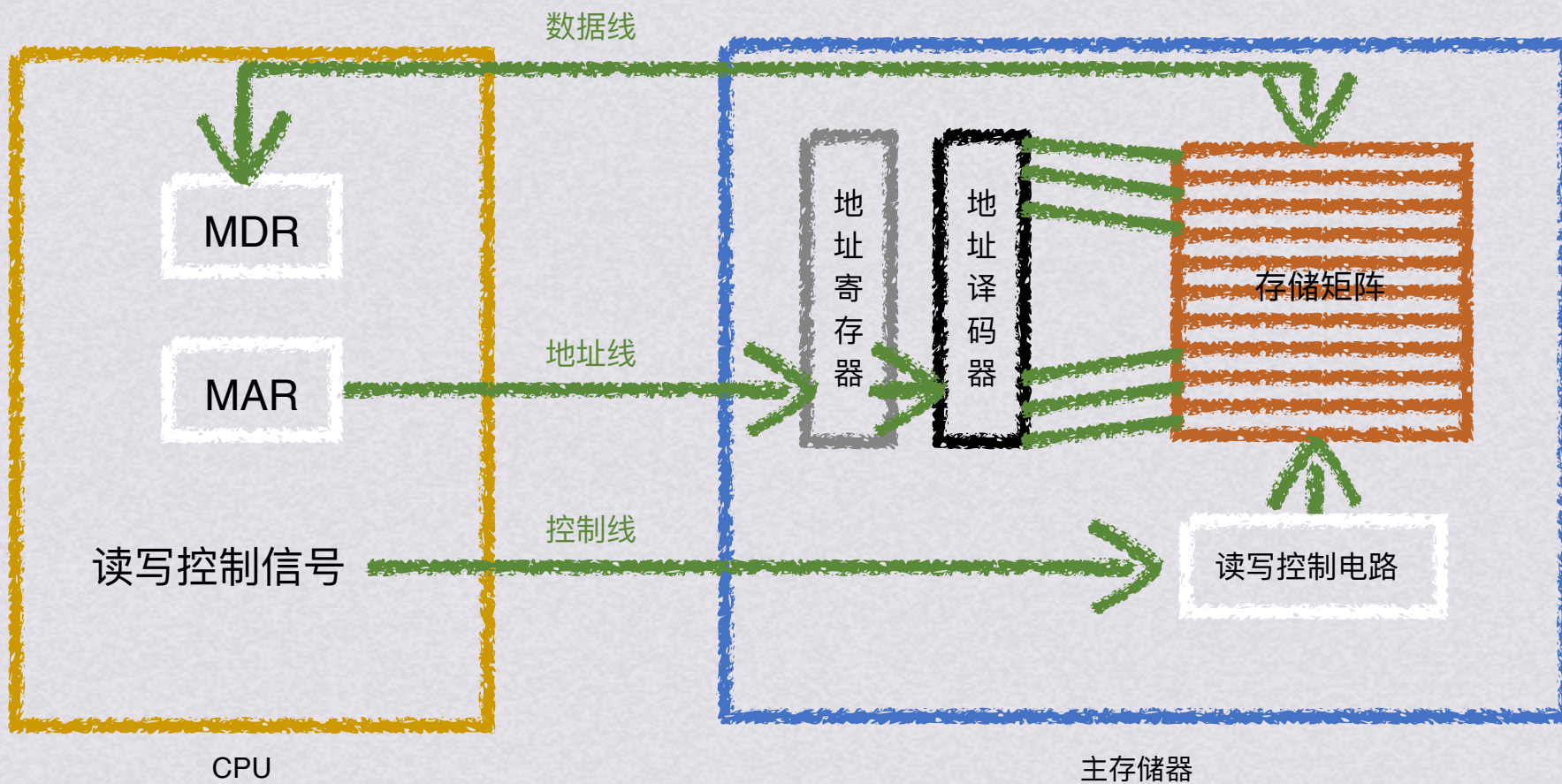


机器指令

3. 指令字长

指令字长

早期计算机中，指令字长=机器字长=存储字长



访问某个存储单元即可取出一条完整的指令

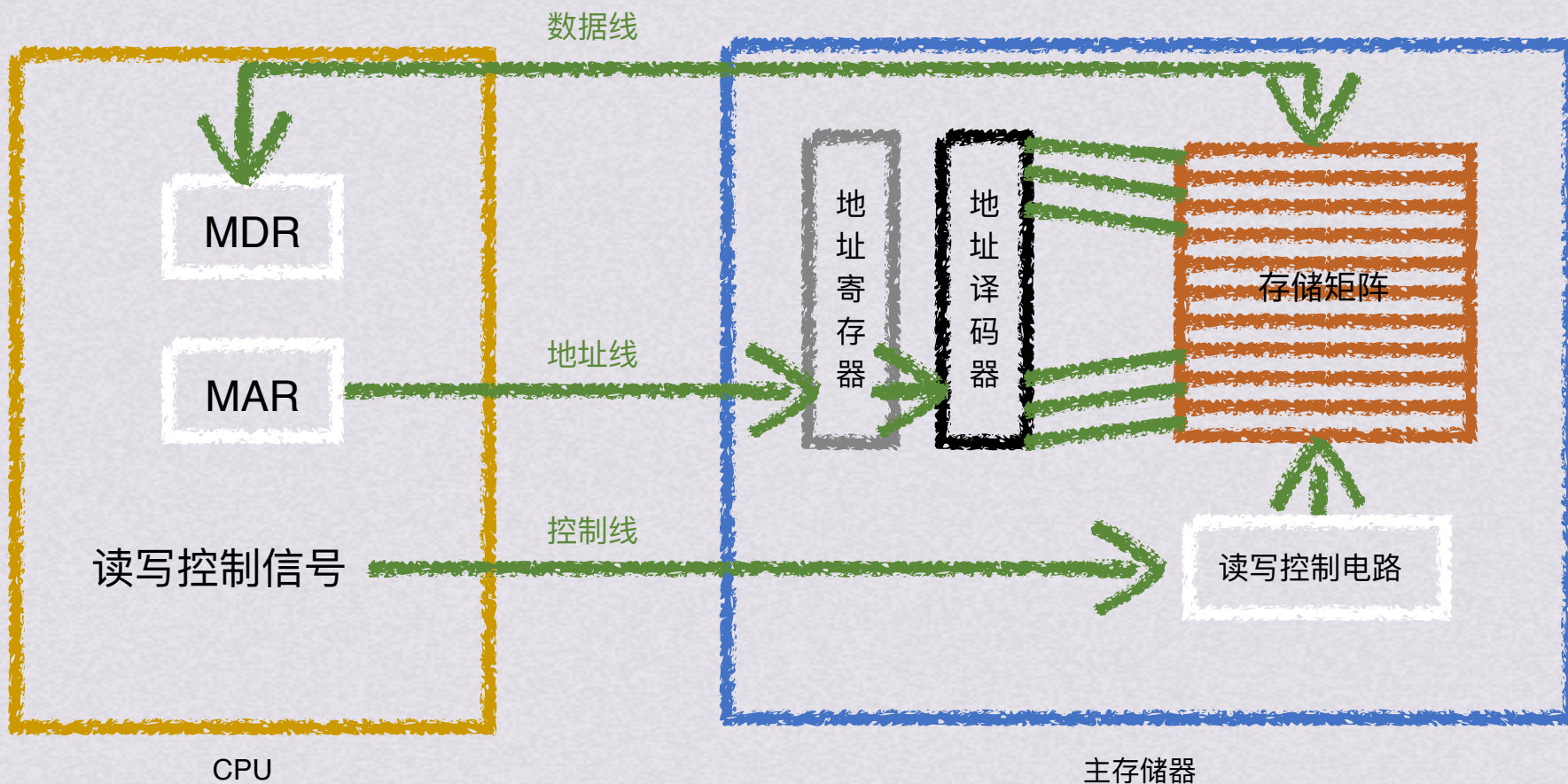


机器指令

3.指令字长

指令字长

随着计算机的发展，现代计算机的机器字长>存储字长，而指令字长取字节的整数倍



如果按字节编址
访问多个存储单元才可取出一
条完整的指令



4.定长操作码与扩展操作码

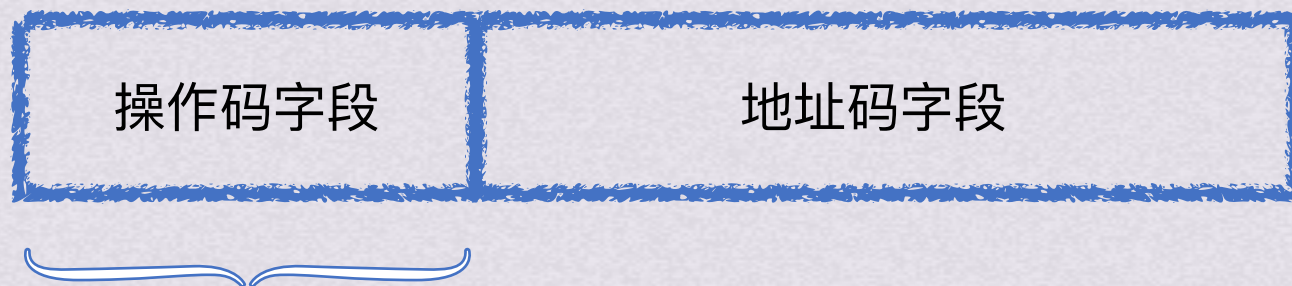


机器指令

4. 定长操作码与扩展操作码

定长操作码

将指令字的最高位部分分配固定的若干位表示操作码



n 位操作码字段的指令系统最大能够表示 2^n 种指令

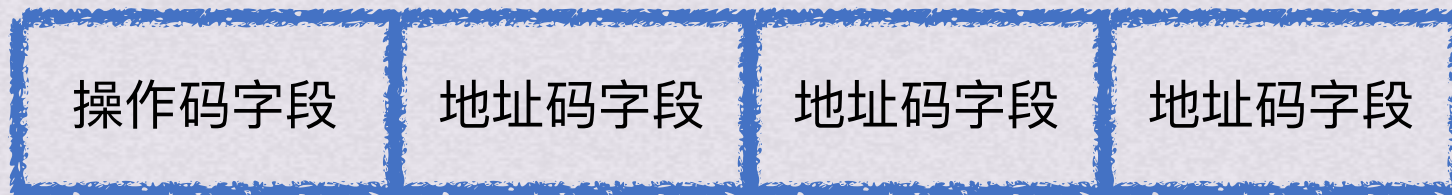


机器指令

4.定长操作码与扩展操作码

扩展操作码

指令的操作码字段的位数不固定，操作码的长度随地址码的减少而增加
不同地址数的指令具有不同长度的操作码，从而在满足需要的前提下有效缩短指令字长





机器指令

4. 定长操作码与扩展操作码

扩展操作码

指令的操作码字段的位数不固定，操作码的长度随地址码的减少而增加
不同地址数的指令具有不同长度的操作码，从而在满足需要的前提下有效缩短指令字长



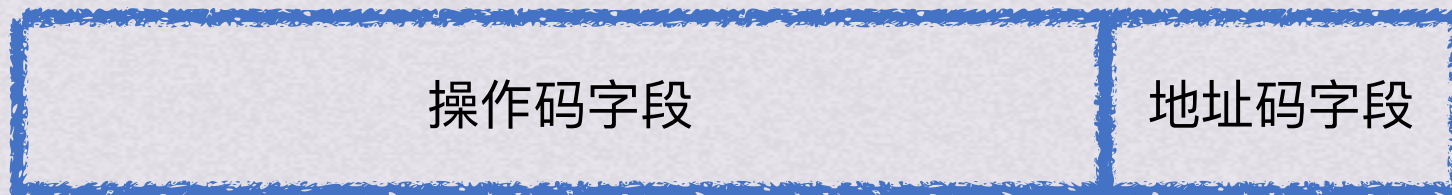


机器指令

4.定长操作码与扩展操作码

扩展操作码

指令的操作码字段的位数不固定，操作码的长度随地址码的减少而增加
不同地址数的指令具有不同长度的操作码，从而在满足需要的前提下有效缩短指令字长





机器指令

4. 定长操作码与扩展操作码

扩展操作码

指令的操作码字段的位数不固定，操作码的长度随地址码的减少而增加
不同地址数的指令具有不同长度的操作码，从而在满足需要的前提下有效缩短指令字长

$OP=A_1=A_2=A_3=4$ 位

4位操作码

OP	A_1	A_2	A_3
0000	A_1	A_2	A_3
0001	A_1	A_2	A_3
...		...	
1110	A_1	A_2	A_3

2^4-1 条三地址指令



机器指令

4.定长操作码与扩展操作码

扩展操作码

指令的操作码字段的位数不固定，操作码的长度随地址码的减少而增加
不同地址数的指令具有不同长度的操作码，从而在满足需要的前提下有效缩短指令字长

4位操作码

2^4-1 条三地址指令

OP		A ₂	A ₃
0000	A ₁	A ₂	A ₃
0001	A ₁	A ₂	A ₃
...	...	...	...
1110	A ₁	A ₂	A ₃
1111	0000	A ₂	A ₃
1111	0001	A ₂	A ₃
...	...	...	...
1111	1110	A ₂	A ₃

为什么三地址指令的操作码没有1111?

8位操作码

2^4-1 条二地址指令



机器指令

4.定长操作码与扩展操作码

扩展操作码

指令的操作码字段的位数不固定，操作码的长度随地址码的减少而增加
不同地址数的指令具有不同长度的操作码，从而在满足需要的前提下有效缩短指令字长

4位操作码

OP		A ₂	A ₃
0000	A ₁	A ₂	A ₃
0001	A ₁	A ₂	A ₃
...	...		
1110	A ₁	A ₂	A ₃

8位操作码

1111	0000	A ₂	A ₃
1111	0001	A ₂	A ₃
...	...		
1111	1110	A ₂	A ₃

4位操作码

OP	A ₁	A ₂	A ₃
----	----------------	----------------	----------------

1111 0000 0000 0000

8位操作码

OP		A ₂	A ₃
----	--	----------------	----------------

1111 0000 0000 0000

对于计算机来说种，这两条指令的机器数完全一致，机器无法区分这是三地址还是二地址指令



机器指令

4.定长操作码与扩展操作码

扩展操作码

指令的操作码字段的位数不固定，操作码的长度随地址码的减少而增加
不同地址数的指令具有不同长度的操作码，从而在满足需要的前提下有效缩短指令字长

OP		A ₂	A ₃
0000	A ₁	A ₂	A ₃
0001	A ₁	A ₂	A ₃
...	...	...	...
1110	A ₁	A ₂	A ₃
1111	0000	A ₂	A ₃
1111	0001	A ₂	A ₃
...	...	...	...
1111	1110	A ₂	A ₃

4位操作码

2⁴-1条三地址指令

留下1111这种编码作为扩展窗口
和后面的4位组成8位的操作码字段
让机器能够识别出这是二地址指令

8位操作码

2⁴-1条二地址指令



机器指令

4. 定长操作码与扩展操作码

扩展操作码

指令的操作码字段的位数不固定，操作码的长度随地址码的减少而增加
不同地址数的指令具有不同长度的操作码，从而在满足需要的前提下有效缩短指令字长

留下1111这种编码作为扩展窗口
和后面的4位组成12位的操作码字段
让机器能够识别出这是一地址指令

	OP		A ₃	
4位操作码				
8位操作码	1111	0000	A ₂	A ₃
	1111	0001	A ₂	A ₃
	...		...	
	1111	1110	A ₂	A ₃
12位操作码	1111	1111	0000	A ₃
	1111	1111	0001	A ₃
	...		...	
	1111	1111	1110	A ₃

2⁴-1条三地址指令

2⁴-1条二地址指令

2⁴-1条一地址指令

机器指令

4.定长操作码与扩展操作码

扩展操作码

指令的操作码字段的位数不固定，操作码的长度随地址码的减少而增加
不同地址数的指令具有不同长度的操作码，从而在满足需要的前提下有效缩短指令字长

4位操作码
8位操作码

12位操作码

留下1111这种编码作为扩展窗口
和后面的4位组成16位的操作码字段
让机器能够识别出这是了零地址指令

16位操作码

OP			A ₃
1111	1111	0000	A ₃
1111	1111	0001	A ₃
...			
1111	1111	1110	A ₃
1111	1111	1111	0000
1111	1111	1111	0001
...			
1111	1111	1111	1111

2⁴-1条三地址指令
2⁴-1条二地址指令

2⁴-1条一地址指令

已经无法继续扩展
因此不需要预留扩展窗口

2⁴条零地址指令



机器指令

4.定长操作码与扩展操作码

扩展操作码

n地址指令的操作码字段拿出若干种编码来表示此时应按n-1地址指令来解释
这些编码叫做扩展窗口



机器指令

4.定长操作码与扩展操作码

扩展操作码 case.1





机器指令

4.定长操作码与扩展操作码

扩展操作码 case.2





机器指令

扩展操作码

4.定长操作码与扩展操作码

假设地址字段为d位，
n地址指令给n-1地址指令留w个扩展窗口
那么n-1地址指令就有 $w \cdot 2^d$ 种



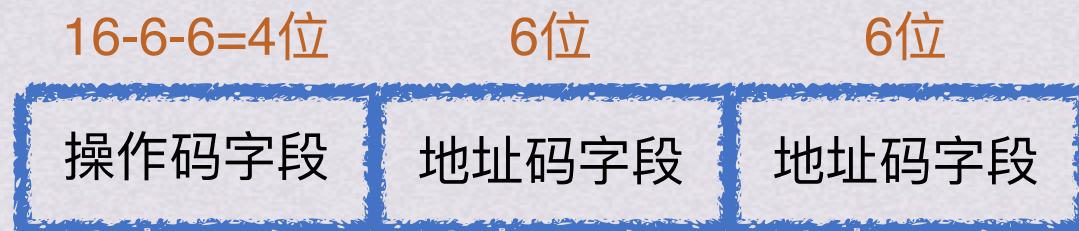


机器指令

4.定长操作码与扩展操作码

【例】假设指令字长为16位，操作数的地址码为6位，指令有零地址、一地址、二地址3种格式。

(1) 设操作码固定，若零地址指令有M种，一地址指令有N种，则二地址指令最多有几种？



操作码占4位，最多可表示 $2^4=16$ 种指令

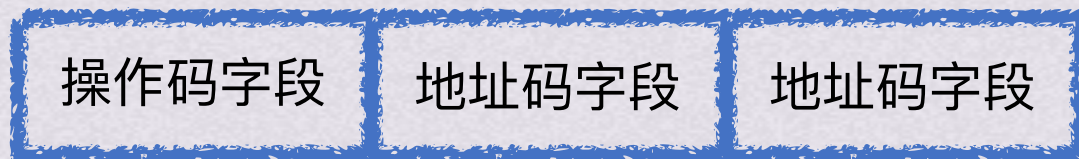


机器指令

4.定长操作码与扩展操作码

【例】假设指令字长为16位，操作数的地址码为6位，指令有零地址、一地址、二地址3种格式。

(1) 设操作码固定，若零地址指令有M种，一地址指令有N种，则二地址指令最多有几种？



4位

6位

6位

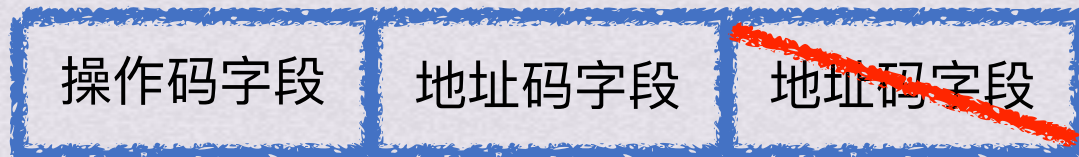
操作码占4位，最多可表示 $2^4=16$ 种指令

零地址指令



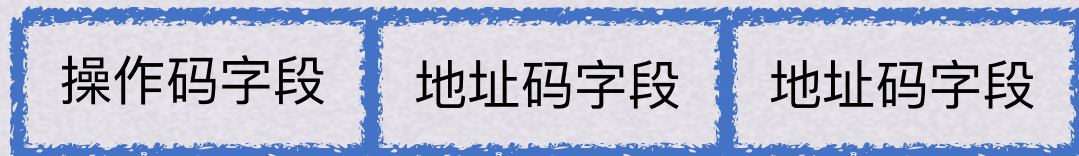
M种

一地址指令



N种

二地址指令



16-M-N种

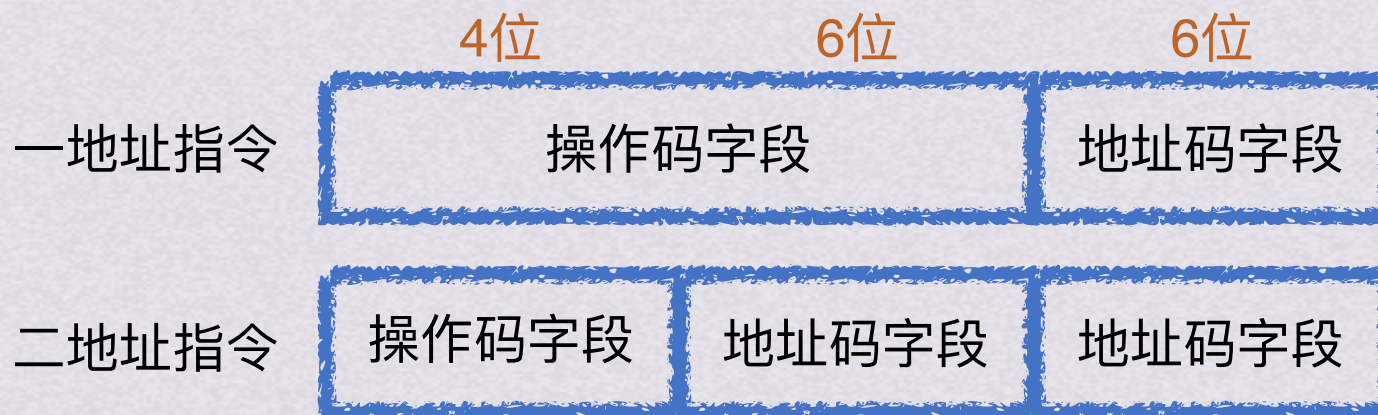


机器指令

4. 定长操作码与扩展操作码

【例】假设指令字长为16位，操作数的地址码为6位，指令有零地址、一地址、二地址3种格式。

(2) 采用扩展操作码技术，二地址指令最多有几种？



二地址指令一共有 2^4 条，必须要留扩展窗口才可以通过扩展操作码得到一地址指令
要想让二地址指令最多，给一地址指令留的扩展窗口就要最少，因此二地址最多有 2^4-1 种

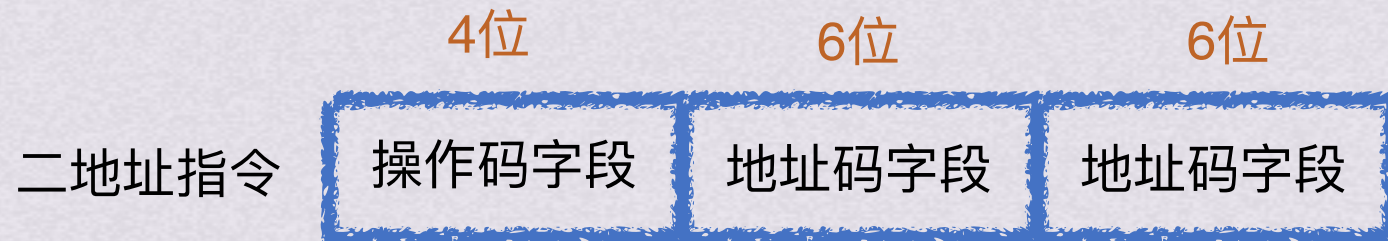


机器指令

4. 定长操作码与扩展操作码

【例】假设指令字长为16位，操作数的地址码为6位，指令有零地址、一地址、二地址3种格式。

(2) 采用扩展操作码技术，若二地址有P条，零地址有Q条，则一地址指令最多有几种？



二地址指令共有 2^{4-w} 种

设二地址指令给一地址指令留了w个扩展窗口

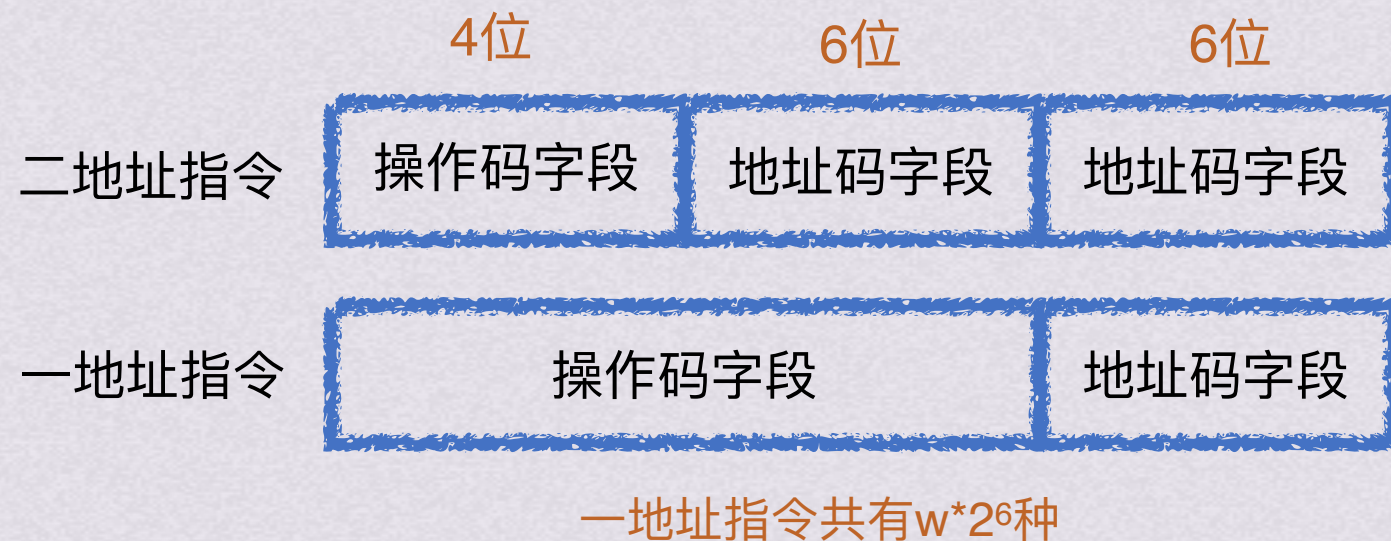


机器指令

4.定长操作码与扩展操作码

【例】假设指令字长为16位，操作数的地址码为6位，指令有零地址、一地址、二地址3种格式。

(2) 采用扩展操作码技术，若二地址有P条，零地址有Q条，则一地址指令最多有几种？



二地址指令共有 $2^4 - w$ 种

设二地址指令给一地址指令留了 w 个扩展窗口

设一地址指令给一地址指令留了 s 个扩展窗口

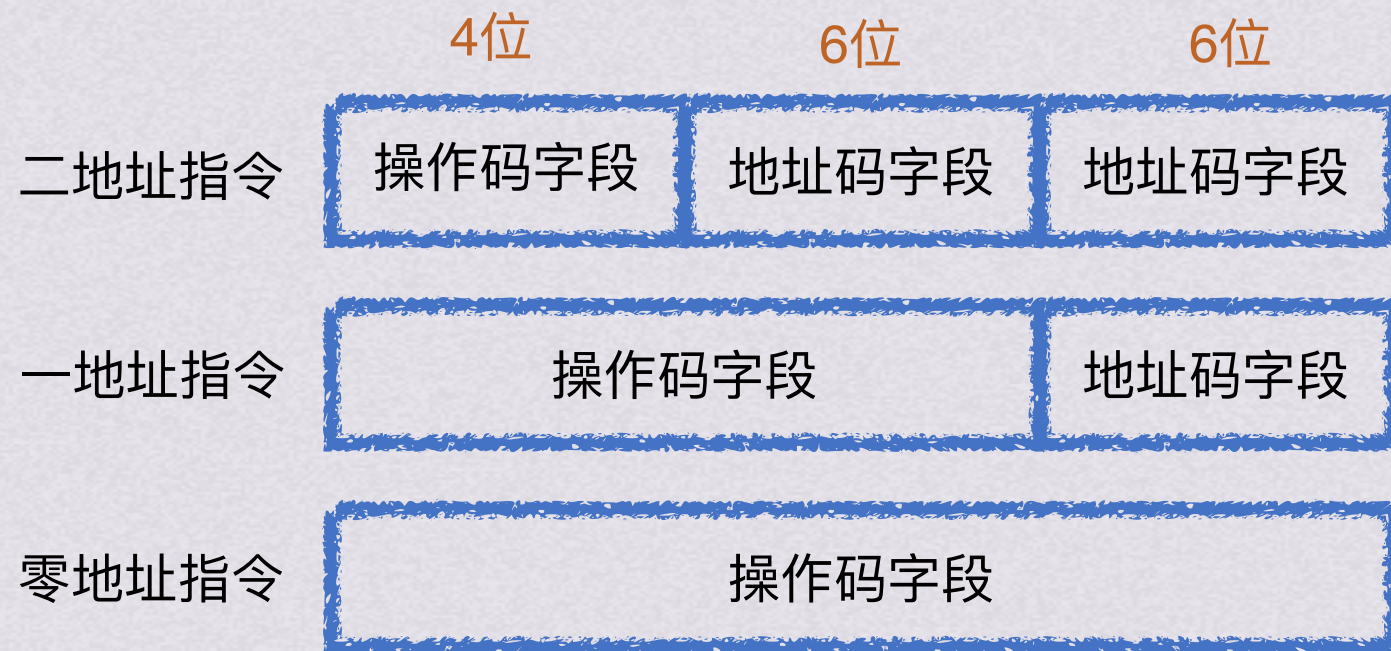


机器指令

4.定长操作码与扩展操作码

【例】假设指令字长为16位，操作数的地址码为6位，指令有零地址、一地址、二地址3种格式。

(2) 采用扩展操作码技术，若二地址有P条，零地址有Q条，则一地址指令最多有几种？



二地址指令共有 2^4-w 种

设二地址指令给一地址指令留了 w 个扩展窗口

一地址指令共有 $w*2^6-s$ 种

设一地址指令给一地址指令留了 s 个扩展窗口

零地址指令共有 $s*2^6=64$ 种

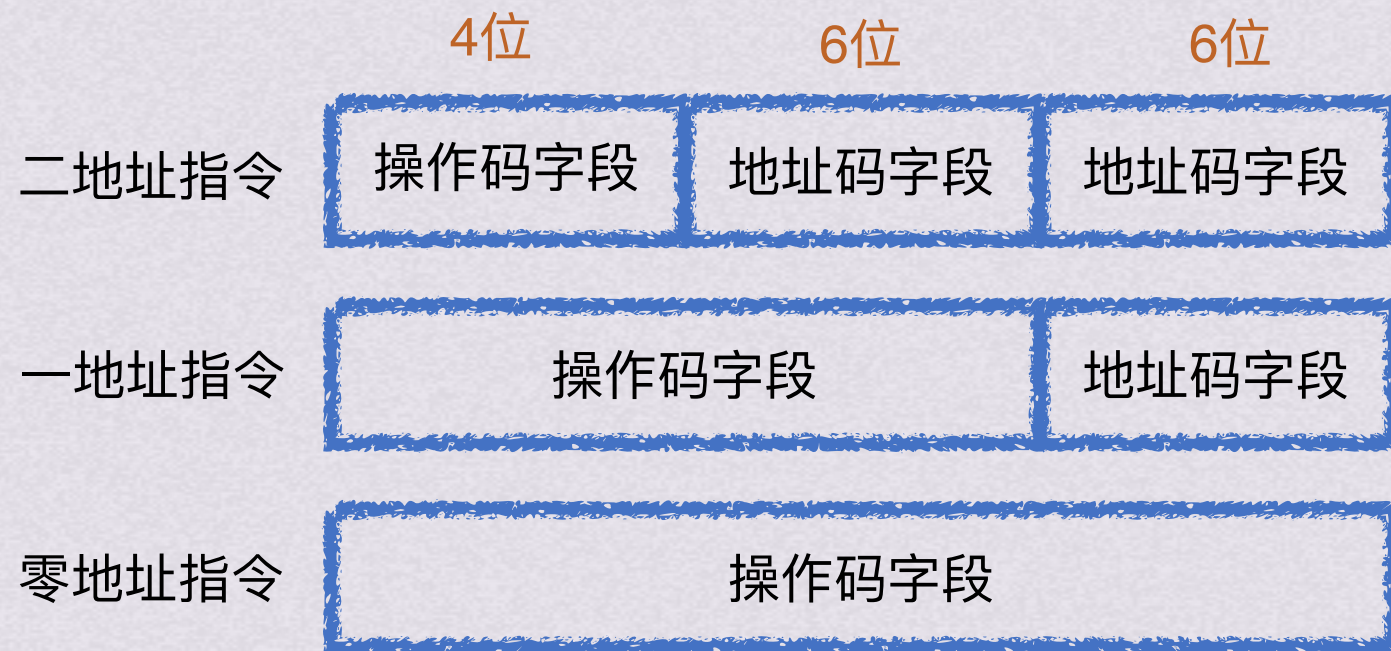


机器指令

4.定长操作码与扩展操作码

【例】假设指令字长为16位，操作数的地址码为6位，指令有零地址、一地址、二地址3种格式。

(2) 采用扩展操作码技术，若二地址有P条，零地址有Q条，则一地址指令最多有几种？



二地址指令共有 $2^4 - w$ 种

设二地址指令给一地址指令留了 w 个扩展窗口

一地址指令共有 $w * 2^6 - s$ 种

设一地址指令给一地址指令留了 s 个扩展窗口

零地址指令共有 $s * 2^6$ 种

不能继续扩展



机器指令

4.定长操作码与扩展操作码

【例】假设指令字长为16位，操作数的地址码为6位，指令有零地址、一地址、二地址3种格式。

(2) 采用扩展操作码技术，若二地址有P条，零地址有Q条，则一地址指令最多有几种？

二地址指令共有 2^4-w 种

设二地址指令给一地址指令留了w个扩展窗口

$$P=2^4-w \longrightarrow w=2^4-P$$

一地址指令共有 $w*2^6-s$ 种

设一地址指令给一地址指令留了s个扩展窗口

因此，一地址指令有 $(2^4-P) * 2^6 - Q / 2^6$ 种

零地址指令共有 $s*2^6$ 种

不能继续扩展

$$Q=s*2^6 \longrightarrow s=Q/2^6$$



5.数据在存储器中的存放方式



机器指令

5.数据在存储器中的存放方式

不边界对齐 数据的起始地址不做限制

按字节编址的32位机器 机器字长是32位，即CPU进行一次整数计算的位数是32位



读出一个字需要2次访存

主存



机器指令

5.数据在存储器中的存放方式

寻址方式

和编址方式不同!

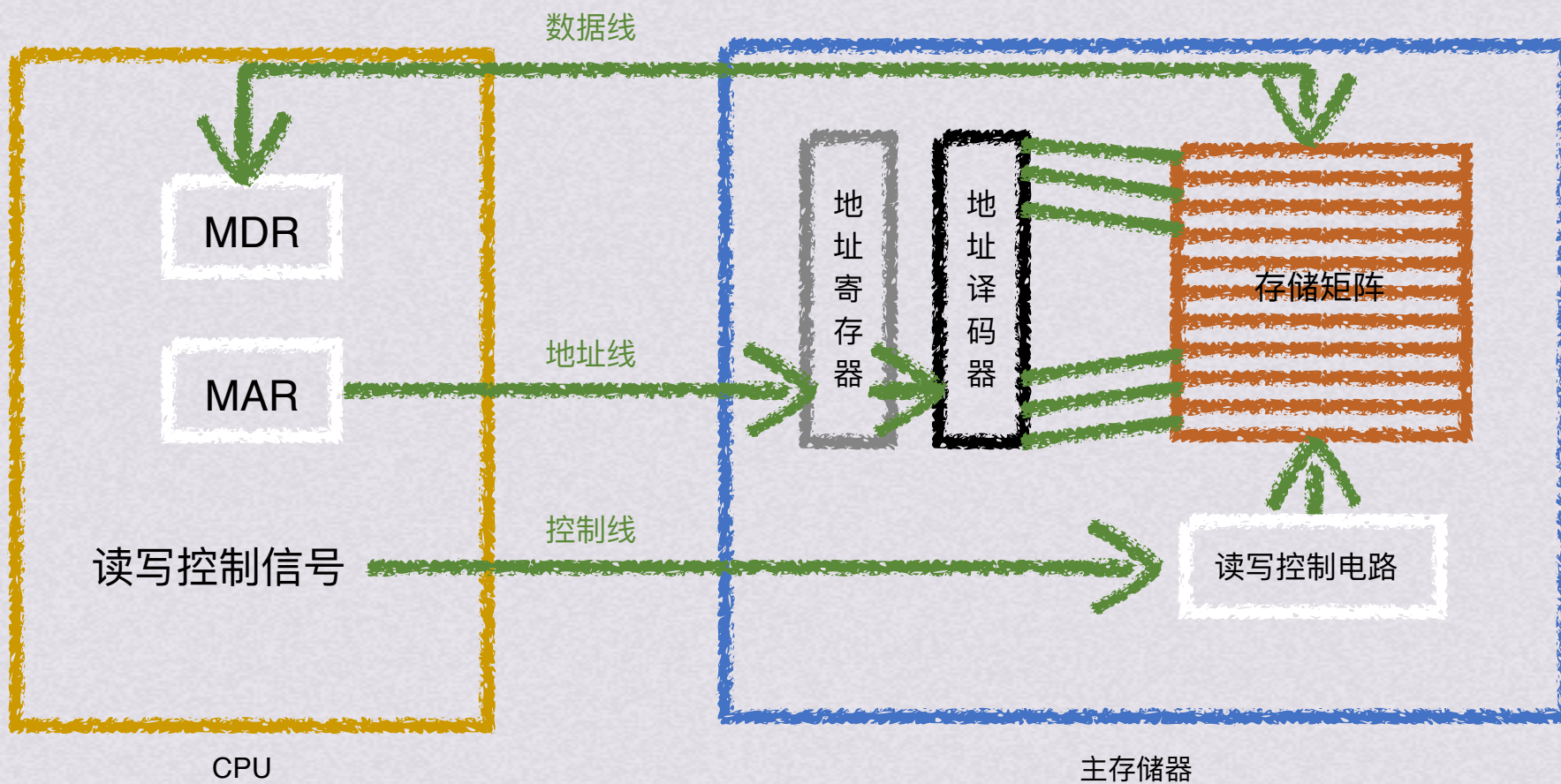
按字节编址的64位计算机

机器字长为64位，即CPU一次能处理的数据宽度是64位，即数据线是64位

由于按字节编址，一个地址只能写8位

因此一次写操作需要重复刚才的过程8次

按字节寻址





机器指令

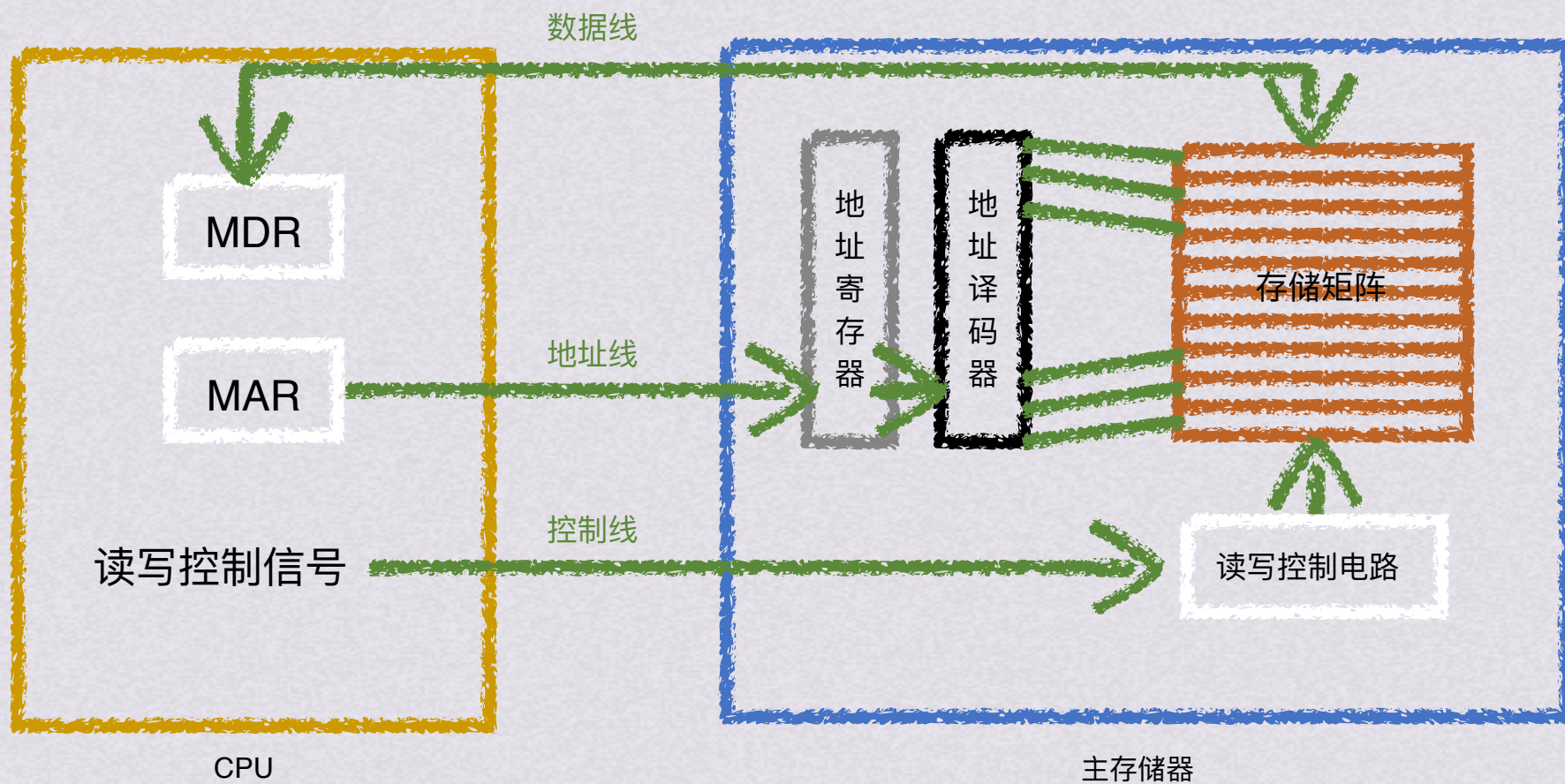
5.数据在存储器中的存放方式

寻址方式

和编址方式不同!

按字节编址的64位计算机

机器字长为64位，即CPU一次能处理的数据宽度是64位，即数据线是64位



按字寻址

一次写操作只需要起始字节的地址就可以读出一个字



机器指令

5.数据在存储器中的存放方式

寻址方式

和编址方式不同!

	编址	寻址
字节	1个字节对应1个地址	1个地址可以读出1个字节
(机器)字	1个字对应1个地址	1个地址可以读出1个字
半(机器)字	1个半字对应1个地址	1个地址可以读出1个半字

大部分情况下,寻址方式默认“按字寻址”



机器指令

5.数据在存储器中的存放方式

边界对齐

K字节大小的数据的起始地址必须是K的整数倍

按字节编址的32位机器 机器字长是32位，即CPU进行一次整数计算的位数是32位



读出一个字只需要1次访存

主存



机器指令

5.数据在存储器中的存放方式

小试牛刀

按字节编址、采用小端方式的32位计算机中，按边界对齐方式为以下C语言结构型变量a分配存储空间：

```
struct record{  
    short x1;  
    int x2;  
}a;
```

若a的首地址为2020 FE00H，a的成员变量x2的机器数为1234 0000H，则其中34H所在的存储单元地址是？



机器指令

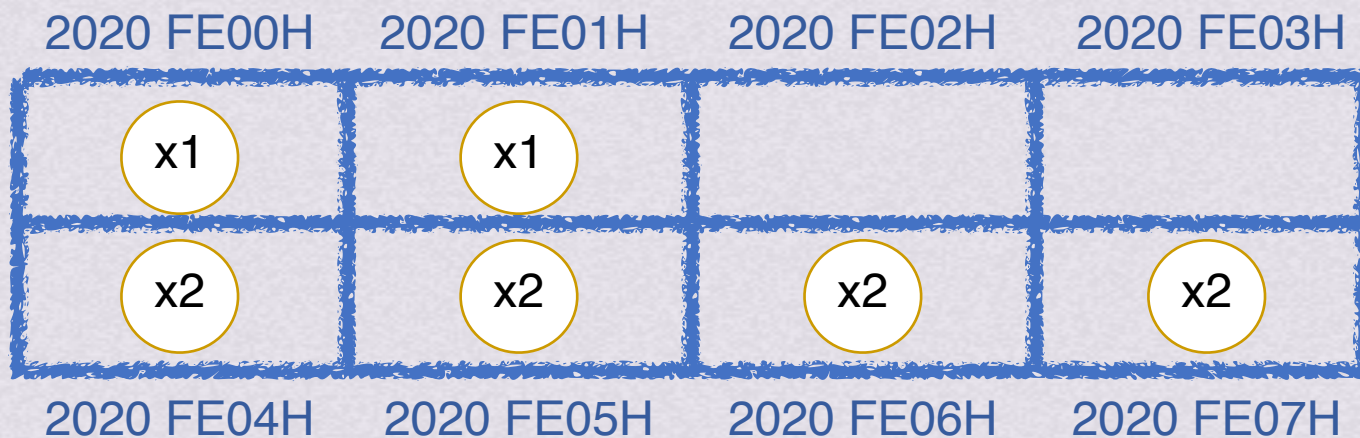
5.数据在存储器中的存放方式

小试牛刀

按字节编址、采用小端方式的32位计算机中，按边界对齐方式为以下C语言结构型变量a分配存储空间：

```
struct record{  
    short x1;  
    int x2;  
}a;
```

若a的首地址为2020 FE00H，a的成员变量x2的机器数为1234 0000H，则其中34H所在的存储单元地址是？





机器指令

5.数据在存储器中的存放方式

小试牛刀

按字节编址、采用小端方式的32位计算机中，按边界对齐方式为以下C语言结构型变量a分配存储空间：

```
struct record{
    short x1;
    int x2;
}a;
```

若a的首地址为2020 FE00H，a的成员变量x2的机器数为1234 0000H，则其中34H所在的存储单元地址是？

